ASSIGNMENT-17.3

Name: Chirra Jyothika

Ht.no:2303A51280

Batch:05

Task 1 Employee Dataset Cleaning

Task Scenario :

Use AI assistance to clean an employee information dataset.

Task Description

Instructions:

- Handle missing values in salary, designation, and experience
- Remove duplicate employee IDs
- Standardize text values (job titles in lowercase)
- Encode categorical features (designation, employment_type)

Prompt:

J Clean an employee dataset using Python (Pandas & Scikit-learn): handle missing values (salary → median, designation → mode, experience → median), remove duplicate employee_id records, convert text columns to lowercase, and encode designation (label encoding) and employment_type (one-hot encoding). Generate clean, complete code with the processed dataset as output.

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# Create a sample DataFrame for demonstration
data = {
    'employee_id': [101, 102, 103, 104, 105, 101, 106, 107, 108, 109],
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Alice', 'Frank', 'Grace', 'Heidi', 'Ivan'],
    'department': ['HR', 'IT', 'Finance', 'IT', 'HR', 'HR', 'Finance', 'IT', 'HR', 'Finance'],
    'designation': ['Manager', 'Developer', 'Analyst', 'Developer', 'Assistant', 'Manager', 'Analyst', 'Developer', 'Manager', 'Analyst'],
    'salary': [70000, 85000, 60000, 90000, 45000, 70000, np.nan, 92000, 75000, 62000],
    'experience': [5, 8, 3, 10, 2, 5, 4, np.nan, 6, 3],
    'employment_type': ['Full-Time', 'Full-Time', 'Contract', 'Full-Time', 'Part-Time', 'Full-Time', 'Contract', 'Full-Time', 'Full-Time', 'Contract']
}
df = pd.DataFrame(data)

print("Original DataFrame:")
display(df)
print("\nDataFrame Info:")
df.info()
```

```
Original DataFrame:
```

	employee_id	name	department	designation	salary	experience	employment_type
0	101	Alice	HR	Manager	70000.0	5.0	Full-Time
1	102	Bob	IT	Developer	85000.0	8.0	Full-Time
2	103	Charlie	Finance	Analyst	60000.0	3.0	Contract
3	104	David	IT	Developer	90000.0	10.0	Full-Time
4	105	Eve	HR	Assistant	45000.0	2.0	Part-Time
5	101	Alice	HR	Manager	70000.0	5.0	Full-Time
6	106	Frank	Finance	Analyst	NaN	4.0	Contract
7	107	Grace	IT	Developer	92000.0	NaN	Full-Time
8	108	Heidi	HR	Manager	75000.0	6.0	Full-Time
9	109	Ivan	Finance	Analyst	62000.0	3.0	Contract

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 7 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   employee_id      10 non-null     int64
 1   name             10 non-null     object
 2   department       10 non-null     object
 3   designation      10 non-null     object
 4   salary           9 non-null      float64
 5   experience       9 non-null      float64
 6   employment_type  10 non-null     object
dtypes: float64(2), int64(1), object(4)
memory usage: 692.0+ bytes
```

Code:

Remove Duplicate employee_id Records:

```
df.drop_duplicates(subset='employee_id', keep='first', inplace=True)

print("DataFrame after removing duplicate employee_id records:")
display(df)
```

```
DataFrame after removing duplicate employee_id records:
```

	employee_id	name	department	designation	salary	experience	employment_type
0	101	Alice	HR	Manager	70000.0	5.0	Full-Time
1	102	Bob	IT	Developer	85000.0	8.0	Full-Time
2	103	Charlie	Finance	Analyst	60000.0	3.0	Contract
3	104	David	IT	Developer	90000.0	10.0	Full-Time
4	105	Eve	HR	Assistant	45000.0	2.0	Part-Time
6	106	Frank	Finance	Analyst	70000.0	4.0	Contract
7	107	Grace	IT	Developer	92000.0	5.0	Full-Time
8	108	Heidi	HR	Manager	75000.0	6.0	Full-Time
9	109	Ivan	Finance	Analyst	62000.0	3.0	Contract

Convert Text Columns to Lowercase:

```
for col in ['name', 'department', 'designation', 'employment_type']:
    if col in df.columns and df[col].dtype == 'object': # Check if column exists and is of object type (string)
        df[col] = df[col].str.lower()

print("DataFrame after converting text columns to lowercase:")
display(df)
```

```
DataFrame after converting text columns to lowercase:
```

	employee_id	name	department	designation	salary	experience	employment_type
0	101	alice	hr	manager	70000.0	5.0	full-time
1	102	bob	it	developer	85000.0	8.0	full-time
2	103	charlie	finance	analyst	60000.0	3.0	contract
3	104	david	it	developer	90000.0	10.0	full-time
4	105	eve	hr	assistant	45000.0	2.0	part-time
6	106	frank	finance	analyst	70000.0	4.0	contract
7	107	grace	it	developer	92000.0	5.0	full-time
8	108	heidi	hr	manager	75000.0	6.0	full-time
9	109	ivan	finance	analyst	62000.0	3.0	contract

Encode Categorical Features:

```
# Label Encoding for 'designation'
le = LabelEncoder()
df['designation_encoded'] = le.fit_transform(df['designation'])

# One-Hot Encoding for 'employment_type'
oh_encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
employment_type_encoded = oh_encoder.fit_transform(df[['employment_type']])
employment_type_df = pd.DataFrame(employment_type_encoded, columns=oh_encoder.get_feature_names_out(['employment_type']))

# Reset index before concatenation if original df was modified with drop_duplicates
df.reset_index(drop=True, inplace=True)

# Concatenate the one-hot encoded columns to the original DataFrame
df = pd.concat([df, employment_type_df], axis=1)

# Drop the original 'employment_type' and 'designation' columns as they are now encoded
df.drop(columns=['employment_type', 'designation'], inplace=True)

print("Final Cleaned and Processed DataFrame:")
display(df)
print("\nDataFrame Info for the cleaned data:")
df.info()
```

Ouput:

```
Final Cleaned and Processed DataFrame:
```

	employee_id	name	department	salary	experience	designation_encoded	employment_type_contract	employment_type_full-time	employment_type_part-time
0	101	alice	hr	70000.0	5.0	3	0.0	1.0	0.0
1	102	bob	it	85000.0	8.0	2	0.0	1.0	0.0
2	103	charlie	finance	60000.0	3.0	0	1.0	0.0	0.0
3	104	david	it	90000.0	10.0	2	0.0	1.0	0.0
4	105	eve	hr	45000.0	2.0	1	0.0	0.0	1.0
5	106	frank	finance	70000.0	4.0	0	1.0	0.0	0.0
6	107	grace	it	92000.0	5.0	2	0.0	1.0	0.0
7	108	heidi	hr	75000.0	6.0	3	0.0	1.0	0.0
8	109	ivan	finance	62000.0	3.0	0	1.0	0.0	0.0

```
DataFrame Info for the cleaned data:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 9 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   employee_id                9 non-null      int64
 1   name                       9 non-null      object
 2   department                 9 non-null      object
 3   salary                     9 non-null      float64
 4   experience                 9 non-null      float64
 5   designation_encoded        9 non-null      int64
 6   employment_type_contract   9 non-null      float64
 7   employment_type_full-time  9 non-null      float64
 8   employment_type_part-time  9 non-null      float64
dtypes: float64(5), int64(2), object(2)
memory usage: 780.0+ bytes
```

Explanation:

- Fill missing values (salary, experience, designation) to avoid incomplete data and ensure accurate analysis.
- Remove duplicate employee IDs and standardize text (lowercase job titles) for consistency.
- Encode categorical features (designation, employment_type) so the dataset can be used in machine learning models.

Task 2: Student Performance Data Preprocessing

Scenario

Preprocess student performance data using AI assistance.

Instructions:

• Convert exam_date column into datetime format

• Extract new features (month, semester, day)

• Normalize total_marks

• Detect and handle outliers in marks using IQR or Z-score

Prompt:

J Preprocess a student performance dataset using Python (Pandas & Scikit-learn): convert exam_date to datetime format, extract new features such as month, semester, and day, normalize total_marks, and detect & handle outliers using IQR or Z-score. Generate clean and complete code with the transformed dataset as output.

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# Create a sample DataFrame for demonstration
data_student = {
    'student_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15],
    'exam_date': ['2023-01-15', '2023-02-20', '2023-03-10', '2023-04-05', '2023-05-12', '2023-09-01', '2023-10-18', '2023-11-25', '2023-12-08', '2023-01-22', '2023-02-14', '2023-03-29',
    'subject': ['Math', 'Science', 'English', 'Math', 'Science', 'English', 'Math', 'Science', 'English', 'Math', 'Science', 'English', 'Math', 'Science', 'English'],
    'grade_level': [10, 11, 10, 12, 11, 10, 12, 11, 10, 11, 12, 10, 11, 12, 10],
    'raw_score': [85, 78, 92, 65, 88, 70, 95, 80, 72, 89, 75, 91, 68, 82, 77],
    'attendance_percentage': [95, 88, 98, 75, 90, 85, 99, 82, 70, 93, 80, 96, 72, 87, 79],
    'total_marks': [88, 80, 90, 68, 85, 73, 93, 83, 75, 91, 78, 94, 70, 85, 80]
}
df_student = pd.DataFrame(data_student)

print("Original Student Performance DataFrame:")
display(df_student)
print("\nDataFrame Info:")
df_student.info()
```

```
Original Student Performance DataFrame:
```

	student_id	exam_date	subject	grade_level	raw_score	attendance_percentage	total_marks
0	1	2023-01-15	Math	10	85	95	88
1	2	2023-02-20	Science	11	78	88	80
2	3	2023-03-10	English	10	92	98	90
3	4	2023-04-05	Math	12	65	75	68
4	5	2023-05-12	Science	11	88	90	85
5	6	2023-09-01	English	10	70	85	73
6	7	2023-10-18	Math	12	95	99	93
7	8	2023-11-25	Science	11	80	82	83
8	9	2023-12-08	English	10	72	70	75
9	10	2023-01-22	Math	11	89	93	91
10	11	2023-02-14	Science	12	75	80	78
11	12	2023-03-29	English	10	91	96	94
12	13	2023-04-10	Math	11	68	72	70
13	14	2023-09-15	Science	12	82	87	85
14	15	2023-10-30	English	10	77	79	80

Code:

- Convert exam_date column into datetime format

```python
# Convert 'exam_date' to datetime format
df_student['exam_date'] = pd.to_datetime(df_student['exam_date'])

# Extract month
df_student['exam_month'] = df_student['exam_date'].dt.month

# Extract day of the week (Monday=0, Sunday=6)
df_student['exam_day_of_week'] = df_student['exam_date'].dt.dayofweek

# Extract semester (example logic: Fall = Sep-Dec, Spring = Jan-May, Summer = Jun-Aug)
def get_semester(month):
    if 9 <= month <= 12: # September, October, November, December
        return 'Fall'
    elif 1 <= month <= 5: # January, February, March, April, May
        return 'Spring'
    else: # June, July, August
        return 'Summer'

df_student['semester'] = df_student['exam_month'].apply(get_semester)

print("DataFrame after date conversion and feature extraction:")
display(df_student)
print("\nDataFrame Info:")
df_student.info()
```

DataFrame after date conversion and feature extraction:

	student_id	exam_date	subject	grade_level	raw_score	attendance_percentage	total_marks	exam_month	exam_day_of_week	semester
0	1	2023-01-15	Math	10	85	95	88	1	6	Spring
1	2	2023-02-20	Science	11	78	88	80	2	0	Spring
2	3	2023-03-10	English	10	92	98	90	3	4	Spring
3	4	2023-04-05	Math	12	65	75	68	4	2	Spring
4	5	2023-05-12	Science	11	88	90	85	5	4	Spring
5	6	2023-09-01	English	10	70	85	73	9	4	Fall
6	7	2023-10-18	Math	12	95	99	93	10	2	Fall
7	8	2023-11-25	Science	11	80	82	83	11	5	Fall
8	9	2023-12-08	English	10	72	70	75	12	4	Fall
9	10	2023-01-22	Math	11	89	93	91	1	6	Spring
10	11	2023-02-14	Science	12	75	80	78	2	1	Spring
11	12	2023-03-29	English	10	91	96	94	3	2	Spring
12	13	2023-04-10	Math	11	68	72	70	4	0	Spring
13	14	2023-09-15	Science	12	82	87	85	9	4	Fall
14	15	2023-10-30	English	10	77	79	80	10	0	Fall

- Extract new features (month, semester, day)

```python
# Convert 'exam_date' to datetime format
df_student['exam_date'] = pd.to_datetime(df_student['exam_date'])

# Extract month
df_student['exam_month'] = df_student['exam_date'].dt.month

# Extract day of the week (Monday=0, Sunday=6)
df_student['exam_day_of_week'] = df_student['exam_date'].dt.dayofweek

# Extract semester (example logic: Fall = Sep-Dec, Spring = Jan-May, Summer = Jun-Aug)
def get_semester(month):
    if 9 <= month <= 12: # September, October, November, December
        return 'Fall'
    elif 1 <= month <= 5: # January, February, March, April, May
        return 'Spring'
    else: # June, July, August
        return 'Summer'

df_student['semester'] = df_student['exam_month'].apply(get_semester)

print("DataFrame after date conversion and feature extraction:")
display(df_student)
print("\nDataFrame Info:")
df_student.info()
```

DataFrame after date conversion and feature extraction:

	student_id	exam_date	subject	grade_level	raw_score	attendance_percentage	total_marks	exam_month	exam_day_of_week	semester	total_marks_normalized	total_marks_capped
0	1	2023-01-15	Math	10	85	95	88	1	6	Spring	0.725605	88
1	2	2023-02-20	Science	11	78	88	80	2	0	Spring	-0.275229	80
2	3	2023-03-10	English	10	92	98	90	3	4	Spring	0.975814	90
3	4	2023-04-05	Math	12	65	75	68	4	2	Spring	-1.776481	68
4	5	2023-05-12	Science	11	88	90	85	5	4	Spring	0.350292	85
5	6	2023-09-01	English	10	70	85	73	9	4	Fall	-1.150960	73
6	7	2023-10-18	Math	12	95	99	93	10	2	Fall	1.351126	93
7	8	2023-11-25	Science	11	80	82	83	11	5	Fall	0.100083	83
8	9	2023-12-08	English	10	72	70	75	12	4	Fall	-0.900751	75
9	10	2023-01-22	Math	11	89	93	91	1	6	Spring	1.100918	91
10	11	2023-02-14	Science	12	75	80	78	2	1	Spring	-0.525438	78
11	12	2023-03-29	English	10	91	96	94	3	2	Spring	1.476231	94
12	13	2023-04-10	Math	11	68	72	70	4	0	Spring	-1.526272	70
13	14	2023-09-15	Science	12	82	87	85	9	4	Fall	0.350292	85
14	15	2023-10-30	English	10	77	79	80	10	0	Fall	-0.275229	80

- Normalize total_marks

```python
# Initialize StandardScaler
scaler = StandardScaler()

# Reshape the column for scaling (StandardScaler expects 2D array)
df_student['total_marks_normalized'] = scaler.fit_transform(df_student[['total_marks']])

print("DataFrame after normalizing 'total_marks':")
display(df_student)
print("\nDescriptive statistics for normalized 'total_marks':")
print(df_student['total_marks_normalized'].describe())
```

DataFrame after normalizing 'total_marks':

	student_id	exam_date	subject	grade_level	raw_score	attendance_percentage	total_marks	exam_month	exam_day_of_week	semester	total_marks_normalized
0	1	2023-01-15	Math	10	85	95	88	1	6	Spring	0.725605
1	2	2023-02-20	Science	11	78	88	80	2	0	Spring	-0.275229
2	3	2023-03-10	English	10	92	98	90	3	4	Spring	0.975814
3	4	2023-04-05	Math	12	65	75	68	4	2	Spring	-1.776481
4	5	2023-05-12	Science	11	88	90	85	5	4	Spring	0.350292
5	6	2023-09-01	English	10	70	85	73	9	4	Fall	-1.150960
6	7	2023-10-18	Math	12	95	99	93	10	2	Fall	1.351126
7	8	2023-11-25	Science	11	80	82	83	11	5	Fall	0.100083
8	9	2023-12-08	English	10	72	70	75	12	4	Fall	-0.900751
9	10	2023-01-22	Math	11	89	93	91	1	6	Spring	1.100918
10	11	2023-02-14	Science	12	75	80	78	2	1	Spring	-0.525438
11	12	2023-03-29	English	10	91	96	94	3	2	Spring	1.476231
12	13	2023-04-10	Math	11	68	72	70	4	0	Spring	-1.526272
13	14	2023-09-15	Science	12	82	87	85	9	4	Fall	0.350292
14	15	2023-10-30	English	10	77	79	80	10	0	Fall	-0.275229

```
Descriptive statistics for normalized 'total_marks':
count    1.500000e+01
mean    -3.404684e-16
std      1.035098e+00
min     -1.776481e+00
25%     -7.130945e-01
50%      1.000834e-01
75%      8.507092e-01
max      1.476231e+00
Name: total_marks_normalized, dtype: float64
```

• Detect and handle outliers in marks using IQR or Z-score

```python
# Calculate Q1 (25th percentile) and Q3 (75th percentile) for 'total_marks'
Q1 = df_student['total_marks'].quantile(0.25)
Q3 = df_student['total_marks'].quantile(0.75)

# Calculate IQR
IQR = Q3 - Q1

# Define outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print(f"Q1: {Q1}, Q3: {Q3}, IQR: {IQR}")
print(f"Lower bound for outliers: {lower_bound}")
print(f"Upper bound for outliers: {upper_bound}")

# Identify outliers
outliers = df_student[(df_student['total_marks'] < lower_bound) | (df_student['total_marks'] > upper_bound)]

print("\nOutliers detected:")
display(outliers)

# Handle outliers by capping (e.g., replace values outside bounds with the bounds)
df_student['total_marks_capped'] = df_student['total_marks'].clip(lower=lower_bound, upper=upper_bound)

print("\nDataFrame after handling outliers in 'total_marks' (capped):")
display(df_student)
print("\nDescriptive statistics for 'total_marks_capped':")
print(df_student['total_marks_capped'].describe())
```

```
Q1: 76.5, Q3: 89.0, IQR: 12.5
Lower bound for outliers: 57.75
Upper bound for outliers: 107.75

Outliers detected:
```

student_id	exam_date	subject	grade_level	raw_score	attendance_percentage	total_marks	exam_month	exam_day_of_week	semester	total_marks_normalized

Output:

```
DataFrame after handling outliers in 'total_marks' (capped):
```

	student_id	exam_date	subject	grade_level	raw_score	attendance_percentage	total_marks	exam_month	exam_day_of_week	semester	total_marks_normalized	total_marks_capped
0	1	2023-01-15	Math	10	85	95	88	1	6	Spring	0.725605	88
1	2	2023-02-20	Science	11	78	88	80	2	0	Spring	-0.275229	80
2	3	2023-03-10	English	10	92	98	90	3	4	Spring	0.975814	90
3	4	2023-04-05	Math	12	65	75	68	4	2	Spring	-1.776481	68
4	5	2023-05-12	Science	11	88	90	85	5	4	Spring	0.350292	85
5	6	2023-09-01	English	10	70	85	73	9	4	Fall	-1.150960	73
6	7	2023-10-18	Math	12	95	99	93	10	2	Fall	1.351126	93
7	8	2023-11-25	Science	11	80	82	83	11	5	Fall	0.100083	83
8	9	2023-12-08	English	10	72	70	75	12	4	Fall	-0.900751	75
9	10	2023-01-22	Math	11	89	93	91	1	6	Spring	1.100918	91
10	11	2023-02-14	Science	12	75	80	78	2	1	Spring	-0.525438	78
11	12	2023-03-29	English	10	91	96	94	3	2	Spring	1.476231	94
12	13	2023-04-10	Math	11	68	72	70	4	0	Spring	-1.526272	70
13	14	2023-09-15	Science	12	82	87	85	9	4	Fall	0.350292	85
14	15	2023-10-30	English	10	77	79	80	10	0	Fall	-0.275229	80

```
Descriptive statistics for 'total_marks_capped':
count    15.000000
mean     82.200000
std       8.273883
min      68.000000
25%      76.500000
50%      83.000000
75%      89.000000
max      94.000000
Name: total_marks_capped, dtype: float64
```

Explanation:

- Convert exam_date to datetime and extract features (month, semester, day) to understand time-based patterns.
- Normalize marks and remove outliers (IQR/Z-score) to improve data quality and model accuracy.

Task 3 -Student Health Data Preparation

Task:

Clean and preprocess student health records using AI scripts.

Instructions:

- Handle missing values in height, weight, BMI
- Encode categorical features (medical_condition, fitness_status)
- Scale numerical features using standard scaling
- Split dataset into training and testing sets

Prompt:

Preprocess a student health dataset using Python (Pandas & Scikit-learn): handle missing values in height, weight, and BMI, encode categorical features (medical_condition, fitness_status), apply standard scaling to numerical features, and split the dataset into training and testing sets. Generate clean and complete code with the final processed dataset as output.

Code:

```python
# Create a sample DataFrame for demonstration
data_health = {
    'student_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'age': [15, 16, 15, 17, 16, 15, 17, 16, 15, 17],
    'height': [160, 175, np.nan, 180, 165, 170, 185, np.nan, 162, 178],
    'weight': [55, 70, 60, 75, np.nan, 68, 80, 72, 58, 73],
    'BMI': [21.4, 22.9, 23.4, np.nan, 22.1, 23.5, 23.3, 22.5, np.nan, 23.0],
    'medical_condition': ['None', 'Asthma', 'None', 'Allergy', 'None', 'Asthma', 'None', 'Allergy', 'None', 'None'],
    'fitness_status': ['Good', 'Excellent', 'Good', 'Fair', 'Good', 'Excellent', 'Excellent', 'Fair', 'Good', 'Good']
}
df_health = pd.DataFrame(data_health)

print("Original Student Health DataFrame:")
display(df_health)
print("\nDataFrame Info:")
df_health.info()
```

```
Original Student Health DataFrame:
```

	student_id	age	height	weight	BMI	medical_condition	fitness_status
0	1	15	160.0	55.0	21.4	None	Good
1	2	16	175.0	70.0	22.9	Asthma	Excellent
2	3	15	NaN	60.0	23.4	None	Good
3	4	17	180.0	75.0	NaN	Allergy	Fair
4	5	16	165.0	NaN	22.1	None	Good
5	6	15	170.0	68.0	23.5	Asthma	Excellent
6	7	17	185.0	80.0	23.3	None	Excellent
7	8	16	NaN	72.0	22.5	Allergy	Fair
8	9	15	162.0	58.0	NaN	None	Good
9	10	17	178.0	73.0	23.0	None	Good

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 7 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   student_id        10 non-null     int64
 1   age               10 non-null     int64
 2   height            8 non-null      float64
 3   weight            9 non-null      float64
 4   BMI               8 non-null      float64
 5   medical_condition 10 non-null     object
 6   fitness_status    10 non-null     object
dtypes: float64(3), int64(2), object(2)
memory usage: 692.0+ bytes
```

- Handle missing values in height, weight, BMI

```python
# Fill missing 'height' with the median
median_height = df_health['height'].median()
df_health['height'].fillna(median_height, inplace=True)

# Fill missing 'weight' with the median
median_weight = df_health['weight'].median()
df_health['weight'].fillna(median_weight, inplace=True)

# Fill missing 'BMI' with the median
median_bmi = df_health['BMI'].median()
df_health['BMI'].fillna(median_bmi, inplace=True)

print("DataFrame after handling missing values:")
display(df_health)
print("\nDataFrame Info after handling missing values:")
df_health.info()
```

```
DataFrame after handling missing values:
/tmp/ipykernel_8986/2990650686.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df_health['height'].fillna(median_height, inplace=True)
/tmp/ipykernel_8986/2990650686.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df_health['weight'].fillna(median_weight, inplace=True)
```

	student_id	age	height	weight	BMI	medical_condition_Allergy	medical_condition_Asthma	medical_condition_None	fitness_status_Excellent	fitness_status_Fair	fitness_status_Good
0	1	-1.083473	-1.581826	-1.727727	-2.290777	0.0	0.0	1.0	0.0	0.0	1.0
1	2	0.120386	0.396456	0.250586	0.163627	0.0	1.0	0.0	1.0	0.0	0.0
2	3	-1.083473	0.065909	-1.068289	0.981761	0.0	0.0	1.0	0.0	0.0	1.0
3	4	1.324244	1.054550	0.910024	0.245440	1.0	0.0	0.0	0.0	1.0	0.0
4	5	0.120386	-0.922732	0.250586	-1.145388	0.0	0.0	1.0	0.0	0.0	1.0
5	6	-1.083473	-0.263638	-0.013189	1.145388	0.0	1.0	0.0	1.0	0.0	0.0
6	7	1.324244	1.713644	1.569462	0.818134	0.0	0.0	1.0	1.0	0.0	0.0
7	8	0.120386	0.065909	0.514381	-0.490881	1.0	0.0	0.0	0.0	1.0	0.0
8	9	-1.083473	-1.318188	-1.332064	0.245440	0.0	0.0	1.0	0.0	0.0	1.0
9	10	1.324244	0.790913	0.646249	0.327254	0.0	0.0	1.0	0.0	0.0	1.0

```
DataFrame Info after handling missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   student_id              10 non-null     int64
 1   age                     10 non-null     float64
 2   height                  10 non-null     float64
 3   weight                  10 non-null     float64
 4   BMI                     10 non-null     float64
 5   medical_condition_Allergy  10 non-null  float64
 6   medical_condition_Asthma   10 non-null  float64
 7   medical_condition_None     10 non-null  float64
 8   fitness_status_Excellent   10 non-null  float64
 9   fitness_status_Fair        10 non-null  float64
 10  fitness_status_Good        10 non-null  float64
dtypes: float64(10), int64(1)
memory usage: 1012.0 bytes
```

- Encode categorical features (medical_condition, fitness_status)

```python
# One-Hot Encoding for 'medical_condition'
oh_encoder_medical = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
medical_condition_encoded = oh_encoder_medical.fit_transform(df_health[['medical_condition']])
medical_condition_df = pd.DataFrame(medical_condition_encoded, columns=oh_encoder_medical.get_feature_names_out(['medical_condition']))

# One-Hot Encoding for 'fitness_status'
oh_encoder_fitness = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
fitness_status_encoded = oh_encoder_fitness.fit_transform(df_health[['fitness_status']])
fitness_status_df = pd.DataFrame(fitness_status_encoded, columns=oh_encoder_fitness.get_feature_names_out(['fitness_status']))

# Reset index before concatenation
df_health.reset_index(drop=True, inplace=True)

# Concatenate the one-hot encoded columns to the original DataFrame
df_health = pd.concat([df_health, medical_condition_df, fitness_status_df], axis=1)

# Drop the original categorical columns
df_health.drop(columns=['medical_condition', 'fitness_status'], inplace=True)

print("DataFrame after encoding categorical features:")
display(df_health)
print("\nDataFrame Info after encoding categorical features:")
df_health.info()
```

```
DataFrame after encoding categorical features:
```

	student_id	age	height	weight	BMI	medical_condition_Allergy	medical_condition_Asthma	medical_condition_None	fitness_status_Excellent	fitness_status_Fair	fitness_status_Good
0	1	15	160.0	55.0	21.40	0.0	0.0	1.0	0.0	0.0	1.0
1	2	16	175.0	70.0	22.90	0.0	1.0	0.0	1.0	0.0	0.0
2	3	15	172.5	60.0	23.40	0.0	0.0	1.0	0.0	0.0	1.0
3	4	17	180.0	75.0	22.95	1.0	0.0	0.0	0.0	1.0	0.0
4	5	16	165.0	70.0	22.10	0.0	0.0	1.0	0.0	0.0	1.0
5	6	15	170.0	68.0	23.50	0.0	1.0	0.0	1.0	0.0	0.0
6	7	17	185.0	80.0	23.30	0.0	0.0	1.0	1.0	0.0	0.0
7	8	16	172.5	72.0	22.50	1.0	0.0	0.0	0.0	1.0	0.0
8	9	15	162.0	58.0	22.95	0.0	0.0	1.0	0.0	0.0	1.0
9	10	17	178.0	73.0	23.00	0.0	0.0	1.0	0.0	0.0	1.0

```
DataFrame Info after encoding categorical features:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   student_id              10 non-null     int64
 1   age                     10 non-null     int64
 2   height                  10 non-null     float64
 3   weight                  10 non-null     float64
 4   BMI                     10 non-null     float64
 5   medical_condition_Allergy  10 non-null  float64
 6   medical_condition_Asthma   10 non-null  float64
 7   medical_condition_None     10 non-null  float64
 8   fitness_status_Excellent   10 non-null  float64
 9   fitness_status_Fair        10 non-null  float64
 10  fitness_status_Good        10 non-null  float64
dtypes: float64(9), int64(2)
memory usage: 1012.0 bytes
```

• Scale numerical features using standard scaling

```
scaler = StandardScaler()
numerical_cols = ['age', 'height', 'weight', 'BMI']
df_health[numerical_cols] = scaler.fit_transform(df_health[numerical_cols])

print("DataFrame after applying standard scaling to numerical features:")
display(df_health)
print("\nDescriptive statistics for scaled numerical features:")
print(df_health[numerical_cols].describe())
```

```
DataFrame after applying standard scaling to numerical features:
```

	student_id	age	height	weight	BMI	medical_condition_Allergy	medical_condition_Asthma	medical_condition_None	fitness_status_Excellent	fitness_status_Fair	fitness_status_Good
0	1	-1.083473	-1.581826	-1.727727	-2.290777	0.0	0.0	1.0	0.0	0.0	1.0
1	2	0.120386	0.395456	0.250586	0.163627	0.0	1.0	0.0	1.0	0.0	0.0
2	3	-1.083473	0.065909	-1.068289	0.981761	0.0	0.0	1.0	0.0	0.0	1.0
3	4	1.324244	1.054550	0.910024	0.245440	1.0	0.0	0.0	0.0	1.0	0.0
4	5	0.120386	-0.922732	0.250586	-1.145388	0.0	0.0	1.0	0.0	0.0	1.0
5	6	-1.083473	-0.263638	-0.013189	1.145388	0.0	1.0	0.0	1.0	0.0	0.0
6	7	1.324244	1.713644	1.569462	0.818134	0.0	0.0	1.0	1.0	0.0	0.0
7	8	0.120386	0.065909	0.514361	-0.490881	1.0	0.0	0.0	0.0	1.0	0.0
8	9	-1.083473	-1.318188	-1.332064	0.245440	0.0	0.0	1.0	0.0	0.0	1.0
9	10	1.324244	0.790913	0.646249	0.327254	0.0	0.0	1.0	0.0	0.0	1.0

```
Descriptive statistics for scaled numerical features:
```

	age	height	weight	BMI
count	1.000000e+01	1.000000e+01	1.000000e+01	1.000000e+01
mean	-4.218847e-16	1.110223e-17	7.105427e-16	-1.781988e-15
std	1.054093e+00	1.054093e+00	1.054093e+00	1.054093e+00
min	-1.083473e+00	-1.581826e+00	-1.727727e+00	-2.290777e+00
25%	-1.083473e+00	-7.579581e-01	-8.045141e-01	-3.272538e-01
50%	1.203859e-01	6.590440e-02	2.505864e-01	2.454403e-01
75%	1.023280e+00	6.920487e-01	6.132772e-01	6.954143e-01
max	1.324244e+00	1.713644e+00	1.569462e+00	1.145388e+00

• Split dataset into training and testing sets

```
# Drop 'student_id' as it's an identifier and not a feature for modeling
X = df_health.drop(columns=['student_id', 'BMI']) # Assuming BMI is what we might want to predict or is excluded for this example
y = df_health['BMI'] # Example target variable

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("\nShape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)

print("\nFirst 5 rows of X_train:")
display(X_train.head())
```

```
Shape of X_train: (8, 9)
Shape of X_test: (2, 9)
Shape of y_train: (8,)
Shape of y_test: (2,)

First 5 rows of X_train:
```

	age	height	weight	medical_condition_Allergy	medical_condition_Asthma	medical_condition_None	fitness_status_Excellent	fitness_status_Fair	fitness_status_Good
5	-1.083473	-0.263638	-0.013189	0.0	1.0	0.0	1.0	0.0	0.0
0	-1.083473	-1.581826	-1.727727	0.0	0.0	1.0	0.0	0.0	1.0
7	0.120386	0.065909	0.514361	1.0	0.0	0.0	0.0	1.0	0.0
2	-1.083473	0.065909	-1.068289	0.0	0.0	1.0	0.0	0.0	1.0
9	1.324244	0.790913	0.646249	0.0	0.0	1.0	0.0	0.0	1.0

Output:

```
print("Final Processed Training Features (X_train) samples:")
display(X_train.head())

print("\nFinal Processed Testing Features (X_test) samples:")
display(X_test.head())

print("\nFinal Processed Training Target (y_train) samples:")
display(y_train.head())
```

```
Final Processed Training Features (X_train) samples:
```

	age	height	weight	medical_condition_Allergy	medical_condition_Asthma	medical_condition_None	fitness_status_Excellent	fitness_status_Fair	fitness_status_Good
5	-1.083473	-0.263638	-0.013189	0.0	1.0	0.0	1.0	0.0	0.0
0	-1.083473	-1.581826	-1.727727	0.0	0.0	1.0	0.0	0.0	1.0
7	0.120386	0.065909	0.514361	1.0	0.0	0.0	0.0	1.0	0.0
2	-1.083473	0.065909	-1.068289	0.0	0.0	1.0	0.0	0.0	1.0
9	1.324244	0.790913	0.646249	0.0	0.0	1.0	0.0	0.0	1.0

```
Final Processed Testing Features (X_test) samples:
```

	age	height	weight	medical_condition_Allergy	medical_condition_Asthma	medical_condition_None	fitness_status_Excellent	fitness_status_Fair	fitness_status_Good
3	-1.083473	-1.318188	-1.332064	0.0	0.0	1.0	0.0	0.0	1.0
1	0.120386	0.395456	0.250586	0.0	1.0	0.0	1.0	0.0	0.0

```
Final Processed Training Target (y_train) samples:
```

	BMI
5	1.145388
0	-2.290777
7	-0.490881
2	0.981761
9	0.327254

```
dtype: float64
```

Explanation:

- Handle missing values and encode categorical features to make the dataset complete and usable for machine learning.
- Apply standard scaling to normalize numerical features like height, weight, and BMI.
- Split the dataset into training and testing sets to build and evaluate predictive models.

Task 4: Real-Time Application: Ride-Sharing Data Cleaning

Scenario

A ride-sharing company has trip data with errors.

Requirements:

• Standardize pickup and drop locations

• Fill missing fare amounts with median values

• Remove duplicate ride entries

• Normalize driver ratings

• Generate a summary of data improvements

Prompt:

J Clean a ride-sharing dataset using Python (Pandas & Scikit-learn): standardize pickup and drop locations, fill missing fare values with median, remove duplicate ride records, normalize driver ratings, and generate a summary of data cleaning steps. Produce clean and complete code with the final processed dataset as output.

Code:

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# Create a sample DataFrame for demonstration
data_rides = {
    'ride_id': [1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009, 1010, 1001],
    'pickup_location': ['Downtown', 'Airport', 'suburbs', 'Downtown', 'Airport', 'City Center', 'Suburbs', 'Downtown', 'AIRPORT', 'City Center', 'Downtown'],
    'dropoff_location': ['Uptown', 'Downtown', 'City Center', 'Uptown', 'Suburbs', 'Airport', 'Downtown', 'Uptown', 'City Center', 'Airport', 'Uptown'],
    'fare': [25.50, np.nan, 18.00, 30.00, 22.00, 15.00, 28.00, np.nan, 40.00, 19.50, 25.50],
    'driver_rating': [4.5, 3.8, 5.0, 4.2, 4.9, 3.5, 4.7, 4.0, np.nan, 4.1, 4.5],
    'ride_date': ['2023-01-01', '2023-01-01', '2023-01-02', '2023-01-02', '2023-01-03', '2023-01-03', '2023-01-04', '2023-01-04', '2023-01-05', '2023-01-05', '2023-01-01']
}
df_rides = pd.DataFrame(data_rides)

print("Original Ride-Sharing DataFrame:")
display(df_rides)
print("\nDataFrame Info:")
df_rides.info()
```

Original Ride-Sharing DataFrame:

	ride_id	pickup_location	dropoff_location	fare	driver_rating	ride_date
0	1001	Downtown	Uptown	25.5	4.5	2023-01-01
1	1002	Airport	Downtown	NaN	3.8	2023-01-01
2	1003	suburbs	City Center	18.0	5.0	2023-01-02
3	1004	Downtown	Uptown	30.0	4.2	2023-01-02
4	1005	Airport	Suburbs	22.0	4.9	2023-01-03
5	1006	City Center	Airport	15.0	3.5	2023-01-03
6	1007	Suburbs	Downtown	28.0	4.7	2023-01-04
7	1008	Downtown	Uptown	NaN	4.0	2023-01-04
8	1009	AIRPORT	City Center	40.0	NaN	2023-01-05
9	1010	City Center	Airport	19.5	4.1	2023-01-05
10	1001	Downtown	Uptown	25.5	4.5	2023-01-01

- Standardize pickup and drop locations

```python
df_rides['pickup_location'] = df_rides['pickup_location'].str.lower()
df_rides['dropoff_location'] = df_rides['dropoff_location'].str.lower()

print("DataFrame after standardizing locations:")
display(df_rides)
```

DataFrame after standardizing locations:

	ride_id	pickup_location	dropoff_location	fare	driver_rating	ride_date
0	1001	downtown	uptown	25.5	4.5	2023-01-01
1	1002	airport	downtown	NaN	3.8	2023-01-01
2	1003	suburbs	city center	18.0	5.0	2023-01-02
3	1004	downtown	uptown	30.0	4.2	2023-01-02
4	1005	airport	suburbs	22.0	4.9	2023-01-03
5	1006	city center	airport	15.0	3.5	2023-01-03
6	1007	suburbs	downtown	28.0	4.7	2023-01-04
7	1008	downtown	uptown	NaN	4.0	2023-01-04
8	1009	airport	city center	40.0	NaN	2023-01-05
9	1010	city center	airport	19.5	4.1	2023-01-05
10	1001	downtown	uptown	25.5	4.5	2023-01-01

- Fill missing fare amounts with median values

```
median_fare = df_rides['fare'].median()
df_rides['fare'].fillna(median_fare, inplace=True)

print("DataFrame after handling missing fare values:")
display(df_rides)
print("\nDataFrame Info after handling missing fare values:")
df_rides.info()
```

```
DataFrame after handling missing fare values:
/tmp/ipykernel_8986/2824776724.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inp

 df_rides['fare'].fillna(median_fare, inplace=True)
```

	ride_id	pickup_location	dropoff_location	fare	driver_rating	ride_date
0	1001	downtown	uptown	25.5	4.5	2023-01-01
1	1002	airport	downtown	25.5	3.8	2023-01-01
2	1003	suburbs	city center	18.0	5.0	2023-01-02
3	1004	downtown	uptown	30.0	4.2	2023-01-02
4	1005	airport	suburbs	22.0	4.9	2023-01-03
5	1006	city center	airport	15.0	3.5	2023-01-03
6	1007	suburbs	downtown	28.0	4.7	2023-01-04
7	1008	downtown	uptown	25.5	4.0	2023-01-04
8	1009	airport	city center	40.0	NaN	2023-01-05
9	1010	city center	airport	19.5	4.1	2023-01-05
10	1001	downtown	uptown	25.5	4.5	2023-01-01

- Remove duplicate ride entries

```
df_rides.drop_duplicates(subset=['ride_id', 'ride_date'], keep='first', inplace=True)

print("DataFrame after removing duplicate ride records:")
display(df_rides)
print("\nDataFrame Info after removing duplicate ride records:")
df_rides.info()
```

```
DataFrame after removing duplicate ride records:
```

	ride_id	pickup_location	dropoff_location	fare	driver_rating	ride_date
0	1001	downtown	uptown	25.5	4.5	2023-01-01
1	1002	airport	downtown	25.5	3.8	2023-01-01
2	1003	suburbs	city center	18.0	5.0	2023-01-02
3	1004	downtown	uptown	30.0	4.2	2023-01-02
4	1005	airport	suburbs	22.0	4.9	2023-01-03
5	1006	city center	airport	15.0	3.5	2023-01-03
6	1007	suburbs	downtown	28.0	4.7	2023-01-04
7	1008	downtown	uptown	25.5	4.0	2023-01-04
8	1009	airport	city center	40.0	NaN	2023-01-05
9	1010	city center	airport	19.5	4.1	2023-01-05

```
DataFrame Info after removing duplicate ride records:
<class 'pandas.core.frame.DataFrame'>
Index: 10 entries, 0 to 9
Data columns (total 6 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   ride_id          10 non-null     int64
 1   pickup_location  10 non-null     object
 2   dropoff_location 10 non-null     object
 3   fare             10 non-null     float64
 4   driver_rating    9 non-null      float64
 5   ride_date        10 non-null     object
dtypes: float64(2), int64(1), object(3)
memory usage: 860.0+ bytes
```

- Normalize driver ratings

```python
# Fill missing 'driver_rating' with the median before scaling
median_driver_rating = df_rides['driver_rating'].median()
df_rides['driver_rating'].fillna(median_driver_rating, inplace=True)

# Initialize StandardScaler
scaler = StandardScaler()

# Reshape the column for scaling (StandardScaler expects 2D array)
df_rides['driver_rating_normalized'] = scaler.fit_transform(df_rides[['driver_rating']])

print("DataFrame after normalizing 'driver_rating':")
display(df_rides)
print("\nDescriptive statistics for normalized 'driver_rating':")
print(df_rides['driver_rating_normalized'].describe())
```

```
DataFrame after normalizing 'driver_rating':
/tmp/ipykernel_8986/2759254728.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform

  df_rides['driver_rating'].fillna(median_driver_rating, inplace=True)
```

	ride_id	pickup_location	dropoff_location	fare	driver_rating	ride_date	driver_rating_normalized
0	1001	downtown	uptown	25.5	4.5	2023-01-01	0.459463
1	1002	airport	downtown	25.5	3.8	2023-01-01	-1.072079
2	1003	suburbs	city center	18.0	5.0	2023-01-02	1.553421
3	1004	downtown	uptown	30.0	4.2	2023-01-02	-0.196913
4	1005	airport	suburbs	22.0	4.9	2023-01-03	1.334629
5	1006	city center	airport	15.0	3.5	2023-01-03	-1.728454
6	1007	suburbs	downtown	28.0	4.7	2023-01-04	0.897046
7	1008	downtown	uptown	25.5	4.0	2023-01-04	-0.634496
8	1009	airport	city center	40.0	4.2	2023-01-05	-0.196913
9	1010	city center	airport	19.5	4.1	2023-01-05	-0.415704

```
Descriptive statistics for normalized 'driver_rating':
count    1.000000e+01
mean    -1.931788e-15
std      1.054093e+00
min     -1.728454e+00
25%     -5.797979e-01
50%     -1.969125e-01
75%      7.876500e-01
max      1.553421e+00
Name: driver_rating_normalized, dtype: float64
```

• Generate a summary of data improvements

Summary of Data Cleaning Steps

Here's a recap of the data cleaning and preprocessing steps performed on the ride-sharing dataset:

- **Standardized Locations**: pickup_location and dropoff_location were converted to lowercase.
- **Handled Missing Fares**: Missing values in the fare column were imputed using the median of the existing fares.
- **Removed Duplicates**: Duplicate ride records were identified and removed based on a combination of ride_id and ride_date.
- **Normalized Driver Ratings**: Missing values in driver_rating were first filled with the median, and then the column was normalized using StandardScaler to have a mean of 0 and standard deviation of 1.

Final Processed Ride-Sharing Dataset

The dataset is now cleaned and preprocessed, ready for further analysis or model building.

Output:

```
print("Final Cleaned and Processed Ride-Sharing DataFrame:")
display(df_rides)
print("\nDataFrame Info for the cleaned data:")
df_rides.info()
```

```
Final Cleaned and Processed Ride-Sharing DataFrame:
```

	ride_id	pickup_location	dropoff_location	fare	driver_rating	ride_date	driver_rating_normalized
0	1001	downtown	uptown	25.5	4.5	2023-01-01	0.459463
1	1002	airport	downtown	25.5	3.8	2023-01-01	-1.072079
2	1003	suburbs	city center	18.0	5.0	2023-01-02	1.553421
3	1004	downtown	uptown	30.0	4.2	2023-01-02	-0.196913
4	1005	airport	suburbs	22.0	4.9	2023-01-03	1.334629
5	1006	city center	airport	15.0	3.5	2023-01-03	-1.728454
6	1007	suburbs	downtown	28.0	4.7	2023-01-04	0.897046
7	1008	downtown	uptown	25.5	4.0	2023-01-04	-0.634496
8	1009	airport	city center	40.0	4.2	2023-01-05	-0.196913
9	1010	city center	airport	19.5	4.1	2023-01-05	-0.415704

```
DataFrame Info for the cleaned data:
<class 'pandas.core.frame.DataFrame'>
Index: 10 entries, 0 to 9
Data columns (total 7 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   ride_id                   10 non-null     int64
 1   pickup_location           10 non-null     object
 2   dropoff_location          10 non-null     object
 3   fare                      10 non-null     float64
 4   driver_rating             10 non-null     float64
 5   ride_date                 10 non-null     object
 6   driver_rating_normalized  10 non-null     float64
dtypes: float64(3), int64(1), object(3)
memory usage: 940.0+ bytes
```

Explanation:

- Standardize locations and remove duplicates to ensure consistency and avoid redundant data.
- Fill missing fare values and normalize driver ratings for accurate analysis.
- Generate a summary of cleaning steps to track data improvements and quality.